API 명세

> Work bridge MVP [P(3-1)] · 작성 2026-10-26 · 근거: `app/api/**/route.ts` 18개 파일(핸들러 24개) · `lib/server/*.ts` · `prisma/schema.prisma` 이 문서는 코드에서 읽히는 것만 적는다. 코드에 없거나 판단이 갈리는 부분은 **코드 확인 필요** 로 표시했다.

---

0. 공통

0.1 기본 URL

```
https://workbridge-gray.vercel.app
```

모든 경로는 `/api/...` 아래. 모든 라우트는 `runtime = "nodejs"`.

0.2 응답 봉투 (`lib/server/api.ts`)

상황	HTTP	본문
성공	200 (생성은 201)	`{ "ok": true, "data": { ... } }`
의도된 실패 (`ApiError`)	라우트가 정한 상태	`{ "ok": false, "code": "…", "message": "…" }`
본문 검증 실패 (`ZodError`)	400	`{ "ok": false, "code": "VALIDATION_FAILED", "message": "<첫 번째 이슈 메시지>", "field": "<첫 번째 이슈 경로>" }`
처리되지 않은 예외	500	`{ "ok": false, "code": "INTERNAL_ERROR", "message": "잠시 후 다시 시도해 주세요" }`

- `VALIDATION_FAILED` 는 zod 이슈 중 **첫 번째 하나만** 돌려준다. `field` 는 `path.join(".")` (예: `"skills"`, 중첩이면 `"a.b"`).
- JSON 파싱 실패는 `400 INVALID_JSON` ("요청 본문을 읽을 수 없습니다").
- 예외: `GET /api/notifications/[id]/open` 은 JSON 이 아니라 **리다이렉트**로 응답한다 (§3.24).
- 날짜 필드(`createdAt` , `deadlineAt` 등)는 Prisma `DateTime` 이 JSON 직렬화된 ISO 8601 문자열(UTC, `Z` 접미)이다.

0.3 인증 (`lib/server/auth.ts` , `lib/server/jwt.ts` , `lib/server/guard.ts`)

자체 JWT. 토큰은 응답 본문에 싣지 않고 **httpOnly 쿠키**로만 오간다. 클라이언트는 `fetch` 에 `credentials: "include"` (같은 오리진이면 기본) 외에 할 일이 없다.

쿠키	내용	수명	비고
`wb_access`	JWT (HS256, `sub` =userId, `role`)	쿠키 `maxAge` 15분 고정. JWT 자체 만료는 env `ACCESS_TOKEN_TTL` (기본 `"15m"`)	검증은 서명만 (DB 조회 없음)
`wb_refresh`	48바이트 난수 (base64url). JWT 아님	env `REFRESH_TOKEN_TTL` (기본 `"14d"` , 정수부 일수로 해석, 파싱 실패 시 14일)	DB `RefreshToken` 에 **sha256 해시** 로 저장. 로그아웃 시 `revokedAt` 기록

공통 쿠키 옵션: `httpOnly` , `sameSite: "lax"` , `path: "/"` , `secure` 는 `NODE_ENV === "production"` 일 때만.

자동 재발급 (AC B-02) — 인증이 필요한 모든 라우트는 `getCurrentUser()` 를 거친다.

1. `wb_access` 가 있고 서명·만료가 유효하면 그대로 통과.
2. 아니면 `wb_refresh` 를 해시해 DB 조회. 없음 / `revokedAt` 있음 / `expiresAt` 지남이면 **비로그인** 취급.
3. 유효하면 **회전(rotation)**: 쓴 refresh 를 즉시 `revokedAt` 처리하고 Access + Refresh 를 새로 발급해 쿠키를 다시 심는다. 따라서 어떤 API 응답에든 `Set-Cookie` 가 붙을 수 있다.

가드 3종

헬퍼	실패 시	쓰는 곳
`requireUser()`	`401 UNAUTHENTICATED` "로그인이 필요합니다"	로그인만 필요한 라우트
`requireRole("client" | "freelancer")`	위 401 또는 `403 ROLE_FORBIDDEN` ("건물주 계정만 이용할 수 있어요" / "전문가 계정만 이용할 수 있어요")	역할 제한 라우트
`requireProfile(user)`	`428 PROFILE_REQUIRED` "먼저 프로필을 작성해 주세요"	프로젝트 등록 · 지원 제출

- 건물주(client)의 프로필 = `ClientProfile` 행 존재, 전문가(freelancer)의 프로필 = `VendorProfile` 행 존재.
- `GET /api/auth/me` 와 `GET /api/notifications/[id]/open` 은 가드를 쓰지 않고 `getCurrentUser()` 를 직접 호출해 비로그인도 200/리다이렉트로 처리한다.

0.4 공통 오류 코드

코드에 실제로 등장하는 `code` 값 전부. HTTP 상태는 코드에서 지정한 값 그대로.

HTTP	code	언제	발생 라우트
400	`VALIDATION_FAILED`	zod 검증 실패 (`field` 동봉)	본문/쿼리를 받는 모든 라우트
400	`INVALID_JSON`	요청 본문이 JSON 이 아님	`readJson` 쓰는 라우트
401	`UNAUTHENTICATED`	세션 없음·만료·무효	인증 필요 라우트 전부
401	`INVALID_CREDENTIALS`	이메일 또는 비밀번호 불일치 (둘을 구분해 주지 않음)	`POST /api/auth/login`
403	`ROLE_FORBIDDEN`	역할이 다름	`requireRole` 라우트
403	`FORBIDDEN`	남의 프로젝트/지원서를 조작	지원 수락·거절, 프로젝트 취소·삭제
404	`NOT_FOUND`	프로젝트/지원서가 없거나 소프트 삭제됨 (남의 프로젝트도 조회 계열은 404)	단건 조회·수정·지원 등

HTTP	code	언제	발생 라우트
409	`EMAIL_TAKEN`	이미 가입된 이메일	`POST /api/auth/signup`
409	`NOT_RECRUITING`	모집 중(`open`)이 아닌 프로젝트에 지원	`POST /api/projects/[id]/applications`
409	`ALREADY_APPLIED`	같은 프로젝트에 중복 지원 (`@@unique([projectId, freelancerId])`)	`POST /api/projects/[id]/applications`
409	`ALREADY_DECIDED`	이미 수락/거절된 지원서	`PATCH /api/applications/[id]`
409	`ALREADY_IN_DEAL`	이미 다른 전문가가 수락돼 거래 중	`PATCH /api/applications/[id]` (accept)
409	`ALREADY_CLOSED`	이미 마감된 프로젝트를 다시 마감	`POST /api/projects/[id]/close`
409	`ALREADY_CANCELLED`	이미 취소된 프로젝트를 다시 취소	`POST /api/projects/[id]/cancel`
409	`CONTRACT_SIGNED`	계약 체결 후(또는 `awaiting_contract` 를 지난 거래 단계) 취소 시도	`POST /api/projects/[id]/cancel`
409	`IN_DEAL`	거래 진행 중인 프로젝트를 수정·삭제	`PATCH` / `DELETE` `/api/projects/[id]`
409	`LOCKED_FIELDS`	지원자가 있는 프로젝트의 예산·마감일 변경	`PATCH /api/projects/[id]`
428	`PROFILE_REQUIRED`	역할별 프로필이 비어 있음	프로젝트 등록, 지원 제출
500	`INTERNAL_ERROR`	그 외 예외 (예: 존재하지 않는 `skillId` 로 FK 위반)	전부

---

1. 엔드포인트 요약

인증 열: `-` 비로그인 가능 · `로그인` 역할 무관 · `건물주` = client 전용 · `전문가` = freelancer 전용 · `+프로필` = `requireProfile` 적용.

#	메서드	경로	인증	설명
1	POST	`/api/auth/signup`	-	회원가입 + 즉시 로그인(쿠키 발급)
2	POST	`/api/auth/login`	-	로그인 (Access/Refresh 쿠키 발급)
3	POST	`/api/auth/logout`	-	로그아웃 (쿠키 삭제 + Refresh 무효화)
4	GET	`/api/auth/me`	-	현재 세션 사용자 (없으면 `user: null`)
5	GET	`/api/skills`	-	전문분야 목록
6	GET	`/api/profile`	로그인	내 역할의 프로필 조회
7	PUT	`/api/profile`	로그인	내 역할의 프로필 저장 (최초·수정 공용, upsert)
8	GET	`/api/projects`	-	프로젝트 목록 (검색·필터·정렬·10건 페이지)
9	POST	`/api/projects`	건물주 +프로필	프로젝트 등록
10	GET	`/api/projects/[id]`	건물주	내 프로젝트 단건 (수정 화면용)
11	PATCH	`/api/projects/[id]`	건물주	내 프로젝트 수정
12	DELETE	`/api/projects/[id]`	건물주	내 프로젝트 소프트 삭제
13	GET	`/api/projects/[id]/applications`	건물주	내 프로젝트의 지원자 목록
14	POST	`/api/projects/[id]/applications`	전문가 +프로필	지원서 제출
15	POST	`/api/projects/[id]/bookmark`	전문가	북마크 토글
16	POST	`/api/projects/[id]/cancel`	건물주	프로젝트 취소 (계약 체결 전)
17	POST	`/api/projects/[id]/close`	건물주	모집 수동 마감
18	PATCH	`/api/applications/[id]`	건물주	지원 수락 / 거절
19	GET	`/api/me/applications`	전문가	내 지원 현황
20	GET	`/api/me/bookmarks`	전문가	내 북마크
21	GET	`/api/me/projects`	건물주	내 프로젝트 목록 (거래 단계 포함)
22	GET	`/api/notifications`	로그인	내 알림 최근 50건 + 미확인 수
23	POST	`/api/notifications`	로그인	알림 읽음 처리 (하나 또는 전부)
24	GET	`/api/notifications/[id]/open`	-	알림 클릭 → 읽음 처리 후 링크로 리다이렉트

> 공개 프로젝트 **상세** 조회 API 는 없다. `/projects/[id]` 상세 페이지(B-10)는 서버 컴포넌트가 DB 를 직접 읽는다. `GET /api/projects/[id]` 는 소유 건물주의 수정 화면 전용이다.

2. 데이터 모양 (여러 응답에서 반복되는 객체)

2.1 `user` (세션 사용자, `loadUser` select)

```
{
  "id": "cm2abc…",
  "email": "owner@example.com",
  "name": "김건물",
  "role": "client",
  "avatarUrl": null,
  "bio": null,
  "clientProfile": { "userId": "cm2abc…" },
  "vendorProfile": null
}
```

`clientProfile` / `vendorProfile` 는 존재 여부만 담는다 (`{ userId }` 또는 `null`). 가입·로그인 응답의 `user` 는 이보다 짧다 (`id, email, name, role` 만).

2.2 프로젝트 카드 (`projectCardSelect` , 평탄화 전)

`GET /api/me/bookmarks` 는 이 모양을 **그대로** 내보내고, `GET /api/projects` 는 §3.8 처럼 평탄화해서 내보낸다. 둘의 모양이 다르다.

```
{
  "id": "cm2p…",
  "title": "옥상 방수 재시공",
  "category": "누수·방수",
  "budget": 3000000,
  "deadlineAt": "2026-11-30T00:00:00.000Z",
  "recruitStatus": "open",
  "createdAt": "2026-10-20T03:12:45.000Z",
  "skills": [{ "skill": { "name": "방수·도장" } }],
  "_count": { "applications": 2 },
  "client": { "name": "김건물", "clientProfile": { "region": "서울 마포구" } }
}
```

---

3. 엔드포인트 상세

3.1 `POST /api/auth/signup` — 회원가입

- 호출: 비로그인
- 요청 본문 (`signupSchema`)

필드	타입	필수	제약
`name`	string	O	trim 후 1~40자
`email`	string	O	이메일 형식, trim + 소문자 변환 후 저장
`password`	string	O	8~72자 (bcrypt 입력 상한)
`role`	`"client"` | `"freelancer"`	O	enum

- 성공 `201`

```
{ "ok": true, "data": { "user": { "id": "cm2…", "email": "owner@example.com", "name": "김건물", "rol
e": "client" } } }
```

- 실패: `400 VALIDATION_FAILED` · `400 INVALID_JSON` · `409 EMAIL_TAKEN`
- 부수 효과: 비밀번호 bcrypt(12 라운드) 해싱 저장 → `issueSession` 으로 `RefreshToken` 행 생성 + `wb_access` / `wb_refresh` 쿠키 설정. 가입 즉시 로그인 상태.

3.2 `POST /api/auth/login` — 로그인

- 호출: 비로그인
- 요청 본문 (`loginSchema`)

필드	타입	필수	제약
`email`	string	O	이메일 형식, trim + 소문자
`password`	string	O	1자 이상

- 성공 `200`

```
{ "ok": true, "data": { "user": { "id": "cm2…", "email": "owner@example.com", "name": "김건물", "rol
e": "client" } } }
```

- 실패: `400 VALIDATION_FAILED` · `401 INVALID_CREDENTIALS` (계정 없음 / 비밀번호 틀림 / `passwordHash` 가 null 인 소셜 전용 계정 모두 같은 응답)
- 부수 효과: `RefreshToken` 행 생성, 쿠키 2개 설정. 기존 세션의 refresh 는 무효화하지 않는다 (여러 기기 동시 로그인 가능).

3.3 `POST /api/auth/logout` — 로그아웃

- 호출: 누구나 (세션 없어도 200)
- 요청 본문: 없음
- 성공 `200`

```
{ "ok": true, "data": { "loggedOut": true } }
```

- 실패: 사실상 없음 (`500 INTERNAL_ERROR` 만)
- 부수 효과: `wb_refresh` 쿠키 값의 해시와 일치하는 미무효 `RefreshToken` 에 `revokedAt` 기록, 쿠키 2개 삭제.

3.4 `GET /api/auth/me` — 현재 세션

- 호출: 누구나
- 성공 `200` (로그인)

```
{
  "ok": true,
  "data": {
    "user": { "id": "cm2…", "email": "pro@example.com", "name": "박전문", "role": "freelancer", "avata
rUrl": null, "bio": null, "clientProfile": null, "vendorProfile": { "userId": "cm2…" } },
    "profileComplete": true
  }
}
```

- 성공 `200` (비로그인) — `profileComplete` 키 자체가 없다

```
{ "ok": true, "data": { "user": null } }
```

- `profileComplete` = 건물주면 `clientProfile` 존재, 전문가면 `vendorProfile` 존재.
- 부수 효과: Access 만료 시 Refresh 회전 + 쿠키 재설정 (§0.3).

3.5 `GET /api/skills` — 전문분야 목록

- 호출: 누구나
- 정렬: `isCustom` 오름차순(기본 분야 먼저) → `name` 오름차순
- 성공 `200`

```
{ "ok": true, "data": { "skills": [ { "id": "cm2s1…", "name": "공조·환기" }, { "id": "cm2s2…", "name":
"방수·도장" } ] } }
```

- 시드 기본 6종: 전기 / 설비·배관 / 방수·도장 / 인테리어·목공 / 소방·안전 / 공조·환기 (`prisma/seed.ts`). `isCustom` 은 응답에 포함되지 않는다.

3.6 `GET /api/profile` — 내 프로필 조회

- 호출: 로그인 (역할 무관, `requireUser`)
- 성공 `200` — 건물주(client)

```
{ "ok": true, "data": { "role": "client", "profile": { "userId": "cm2…", "buildingName": "마포타워", "buildingType": "오피스", "region": "서울 마포구", "website": null } } }
```

- 성공 `200` — 전문가(freelancer). `skills` 는 **skill id 배열**

```
{ "ok": true, "data": { "role": "freelancer", "profile": { "userId": "cm2…", "mainCategory": "방수·도장", "years": 12, "hourlyRate": 50000, "portfolioUrl": "", "skills": ["cm2s2…", "cm2s4…"] } } }
```

- 프로필이 아직 없으면 `profile: null`.
- 실패: `401 UNAUTHENTICATED`

3.7 `PUT /api/profile` — 내 프로필 저장

- 호출: 로그인. 세션의 `role` 에 따라 스키마가 갈린다 (본문에 role 을 보내지 않는다).
- 요청 본문 — 건물주 (`clientProfileSchema`)

필드	타입	필수	제약
`buildingName`	string	O	trim 후 1~60자
`buildingType`	`"상가"` | `"오피스"` | `"다세대"` | `"기타"`	O	enum
`region`	string	O	trim 후 1~40자
`website`	string	X	URL 형식 또는 빈 문자열 `""`

- 요청 본문 — 전문가 (`vendorProfileSchema`)

필드	타입	필수	제약
`mainCategory`	string	O	trim 후 1자 이상 (enum 강제 없음)
`years`	number (문자열도 coerce)	O	정수 0~70
`hourlyRate`	number (coerce)	O	정수 0 이상
`portfolioUrl`	string	X	URL 형식 또는 `""`
`skills`	string[]	O	skill **id** 배열, 1개 이상, 각 원소 trim 후 1자 이상

- 성공 `200` — 건물주: 저장된 `ClientProfile` 행

```
{ "ok": true, "data": { "profile": { "userId": "cm2…", "buildingName": "마포타워", "buildingType": "오피스", "region": "서울 마포구", "website": "" } } }
```

- 성공 `200` — 전문가: 저장된 `VendorProfile` 행. **`skills` 는 응답에 없다** (GET 과 모양이 다름)

```
{ "ok": true, "data": { "profile": { "userId": "cm2…", "mainCategory": "방수·도장", "years": 12, "hourlyRate": 50000, "portfolioUrl": "" } } }
```

- 실패: `401 UNAUTHENTICATED` · `400 VALIDATION_FAILED` · `400 INVALID_JSON` · 존재하지 않는 skill id → FK 위반으로 `500 INTERNAL_ERROR` (사전 검증 없음)
- 부수 효과: upsert. 전문가는 트랜잭션 안에서 `VendorSkill` 전부 삭제 후 재생성(통째 교체). `website` / `portfolioUrl` 에 `""` 를 보내면 `""` 로 저장된다(null 변환 없음). 필드를 아예 빼면 upsert `update` 에서 건드리지 않는다.

3.8 `GET /api/projects` — 프로젝트 목록

- 호출: 누구나
- 쿼리 파라미터 (`projectQuerySchema`). `Object.fromEntries(searchParams)` 를 그대로 검증하므로 **빈 문자열도 값으로 들어간다** (아래 비고).

파라미터	타입	기본값	허용값·제약	where 효과 (`buildProjectWhere`)
`q`	string	-	trim 후 최대 60자	`title` 또는 `description` 에 대소문자 무시 부분일치 (`OR`)
`category`	string	-	-	`category =` 정확히 일치
`skill`	string	-	skill **이름** (id 아님)	`skills.some(skill.name = )`
`budgetMin`	number (coerce)	-	정수 0 이상	`budget >=`
`budgetMax`	number (coerce)	-	정수 0 이상	`budget <=`
`deadlineBefore`	ISO 8601 datetime	-	`z.iso.datetime()`	`deadlineAt <=`
`status`	`scheduled` | `open` | `closed`	-	enum	`recruitStatus =`
`sort`	`latest` | `deadline` | `budget`	`latest`	enum	아래 정렬표
`page`	number (coerce)	1	정수 1 이상	`skip = (page-1)*10` , `take = 10`

항상 `deletedAt: null` 조건이 붙는다. `dealStatus` 는 필터도 노출도 하지 않는다 (B-17).

정렬 (`buildProjectOrderBy`)

sort	orderBy
`latest`	`createdAt desc`
`deadline`	`deadlineAt asc` , `createdAt desc`
`budget`	`budget desc (null 은 맨 뒤)` , `createdAt desc`

- 비고: `?status=` / `?sort=` / `?page=` 처럼 **빈 값**을 보내면 enum·min(1) 검증에 걸려 `400 VALIDATION_FAILED` 가 난다. `?q=` `?category=` `?skill=` 의 빈 값은 falsy 로 취급돼 필터가 빠진다. `?budgetMin=` 빈 값은 `0` 으로 coerce 돼 `budget >= 0` 필터가 붙는다(예산 미정 `null` 이 제외됨).
- 성공 `200`

```
{
  "ok": true,
  "data": {
    "projects": [
      {
        "id": "cm2p…",
        "title": "옥상 방수 재시공",
        "category": "누수·방수",
        "budget": 3000000,
        "deadlineAt": "2026-11-30T00:00:00.000Z",
        "recruitStatus": "open",
        "skills": ["방수·도장"],
        "applicationCount": 2,
        "clientName": "김건물",
        "region": "서울 마포구"
      }
    ],
    "page": 1,
    "pageSize": 10,
    "total": 23,
    "totalPages": 3
  }
}
```


`totalPages` 는 최소 1. `region` 은 건물주 프로필이 없으면 `null`. `applicationCount` 는 상태 무관 전체 지원 수.

- 실패: `400 VALIDATION_FAILED`
- 부수 효과: 조회 전에 `syncRecruitStatuses()` 실행 → ① `scheduled` 이고 `recruitStartAt <= now` 인 프로젝트를 `open` 으로, ② `closed` 가 아니고 `deadlineAt < now` 인 프로젝트를 각각 트랜잭션으로 **마감 처리**(대기 지원 일괄 거절 + `recruit_closed` 알림, §3.17 과 동일). 읽기 요청이지만 DB 를 쓴다.

3.9 `POST /api/projects` — 프로젝트 등록

- 호출: 건물주(client) + 프로필 필수
- 요청 본문 (`createProjectSchema` = step1 + step2 + step3)

필드	타입	필수	제약
`title`	string	O	trim 후 5~80자
`description`	string	O	trim 후 20~4000자
`category`	string	O	trim 후 1자 이상. `PROJECT_CATEGORIES` 8종(부록 A.5)이 상수로 있으나 **서버는 enum 으로 강제하지 않는다**
`recruitStartAt`	ISO 8601 datetime | null	X	없음/null 이면 즉시 모집
`deadlineAt`	ISO 8601 datetime	O	메시지 "마감일을 선택해 주세요"

필드	타입	필수	제약
`budget`	number (coerce) | null	X	정수 0 이상. 없음/null → `null` (예산 미정)
`skills`	string[]	O	skill **id** 배열 1개 이상

- 모집 상태 결정: `recruitStartAt` 이 없거나 `<= now` 면 `open` , 미래면 `scheduled` .
- 성공 `201`


```
{ "ok": true, "data": { "project": { "id": "cm2p…", "title": "옥상 방수 재시공", "recruitStatus": "ope
n" } } }
```


- 실패: `401 UNAUTHENTICATED` · `403 ROLE_FORBIDDEN` · `428 PROFILE_REQUIRED` · `400 VALIDATION_FAILED` · `400 INVALID_JSON` · 잘못된 skill id → `500 INTERNAL_ERROR`
- 검증 순서: 역할 → 프로필 → 본문. 프로필이 없으면 본문이 틀려도 428 이 먼저.
- 부수 효과: `Project` + `ProjectSkill` 생성 (nested create). `deadlineAt` 이 과거여도 등록은 막지 않는다 — 다음 목록 조회 때 자동 마감된다.

3.10 `GET /api/projects/[id]` — 내 프로젝트 단건 (수정 화면용)

- 호출: 건물주(client). 본인 소유 + 미삭제만.
- 경로: `id` — 프로젝트 id (cuid)
- 성공 `200` — `Project` 전체 컬럼 + 가공 3개

```
{
  "ok": true,
  "data": {
    "project": {
      "id": "cm2p…",
      "clientId": "cm2…",
      "title": "옥상 방수 재시공",
      "description": "지난 장마 때 …",
      "category": "누수·방수",
      "recruitStartAt": null,
      "deadlineAt": "2026-11-30T00:00:00.000Z",
      "budget": 3000000,
      "recruitStatus": "open",
      "dealStatus": null,
      "deletedAt": null,
      "createdAt": "2026-10-20T03:12:45.000Z",
      "_count": { "applications": 2 },
      "skills": ["cm2s2…"],
      "applicationCount": 2,
      "lockedFields": ["budget", "deadlineAt"]
    }
  }
}
```

`lockedFields` 는 지원자가 1명 이상이면 `["budget", "deadlineAt"]`, 아니면 `[]`. `_count` 는 spread 로 남아 있다.

- 실패: `401` · `403 ROLE_FORBIDDEN` · `404 NOT_FOUND` (남의 프로젝트도 404)
- 부수 효과: 없음 (`syncRecruitStatuses` 호출 안 함).

3.11 `PATCH /api/projects/[id]` — 내 프로젝트 수정

- 호출: 건물주(client), 본인 소유 + 미삭제
- 요청 본문 (`updateProjectSchema` = `createProjectSchema.partial()`): §3.9 의 모든 필드가 선택. 보낸 필드만 갱신.
- `recruitStartAt: null` 을 명시하면 시작일을 지우고 즉시 모집(`open`)으로.
- `skills` 를 보내면 `ProjectSkill` 전부 삭제 후 재생성(통째 교체).
- 규칙 1. `dealStatus` 가 있고 `cancelled` 가 아니면 → `409 IN_DEAL` (수정 자체 불가) 2. 지원자가 1명 이상이면 `budget` / `deadlineAt` 은 **기존 값과 달라질 때만** `409 LOCKED_FIELDS`. 같은 값을 다시 보내는 건 통과. 3. `recruitStartAt` 을 보냈고 현재 `recruitStatus !== "closed"` 면 상태 재계산 (`!startAt || startAt <= now` → `open`, 아니면 `scheduled`). **`closed` 는 수정으로 되살리지 않는다.**
- 성공 `200`

```
{ "ok": true, "data": { "project": { "id": "cm2p…", "title": "옥상 방수 재시공(수정)", "recruitStatus": "open" } } }
```

- 실패: `401` · `403 ROLE_FORBIDDEN` · `404 NOT_FOUND` · `400 VALIDATION_FAILED` · `409 IN_DEAL` · `409 LOCKED_FIELDS` · 잘못된 skill id → `500`
- 검증 순서: 역할 → 본문 zod → 존재/소유 → IN_DEAL → LOCKED_FIELDS
- 부수 효과: 트랜잭션 (skills 교체 + project update). 알림 없음.

3.12 `DELETE /api/projects/[id]` — 소프트 삭제

- 호출: 건물주(client), 본인 소유 + 미삭제
- 요청 본문: 없음
- 규칙: `dealStatus` 가 있고 `cancelled` 가 아니면 `409 IN_DEAL` ("먼저 취소해 주세요")
- 성공 `200`

```
{ "ok": true, "data": { "projectId": "cm2p…" } }
```

- 실패: `401` · `403 ROLE_FORBIDDEN` · `404 NOT_FOUND` (없음/이미 삭제됨) · `403 FORBIDDEN` (남의 프로젝트 — `softDeleteProject` 는 404 가 아니라 403) · `409 IN_DEAL`
- 부수 효과 (한 트랜잭션): `pending` 지원 전부 `rejected` → 프로젝트 `deletedAt = now` , `recruitStatus = closed` → 거절된 지원자에게 `project_cancelled` 알림. 행은 물리 삭제하지 않으며 `Application` · `Bookmark` 도 남는다 (목록/북마크 조회에서는 `deletedAt: null` 로 걸러짐, 내 지원 현황에서는 `project.deletedAt` 으로 표시됨).

3.13 `GET /api/projects/[id]/applications` — 지원자 목록

- 호출: 건물주(client), 본인 소유 + 미삭제
- 정렬: `status asc` , `createdAt asc` . 코드 주석은 "accepted → pending → rejected" 라고 적혀 있으나 PostgreSQL enum 정렬은 선언 순(`pending, accepted, rejected`)을 따를 수 있음 — **코드 확인 필요**.
- 성공 `200`

```
{
  "ok": true,
  "data": {
    "project": { "id": "cm2p…", "title": "옥상 방수 재시공", "recruitStatus": "open", "dealStatus": nul
l },
    "applications": [
      {
        "id": "cm2a…",
        "status": "pending",
        "coverLetter": "옥상 전면 우레탄 …",
        "proposedAmount": 2800000,
        "proposedDays": 7,
        "createdAt": "2026-10-21T01:00:00.000Z",
        "freelancer": {
          "id": "cm2…",
          "name": "박전문",
          "avatarUrl": null,
          "bio": null,
          "mainCategory": "방수·도장",
          "years": 12,
          "hourlyRate": 50000,
          "portfolioUrl": "",
          "skills": ["방수·도장", "설비·배관"]
        }
      }
    ]
  }
}
```

`freelancer.skills` 는 **이름** 배열. 전문가 프로필이 없으면 `mainCategory` / `years` / `hourlyRate` / `portfolioUrl` 은 `null`, `skills` 는 `[]`.

- 실패: `401` · `403 ROLE_FORBIDDEN` · `404 NOT_FOUND`
- 부수 효과: 없음

3.14 `POST /api/projects/[id]/applications` — 지원서 제출

- 호출: 전문가(freelancer) + 프로필 필수
- 요청 본문 (`applySchema`)

필드	타입	필수	제약
`coverLetter`	string	O	trim 후 20~2000자
`proposedAmount`	number (coerce)	O	정수 10,000 ~ 1,000,000,000 (원)
`proposedDays`	number (coerce)	O	정수 1 ~ 365 (일)

- 규칙: 프로젝트가 존재·미삭제이고 `recruitStatus === "open"` 이어야 한다. `scheduled` 면 메시지 "아직 모집 전이에요", `closed` 면 "모집이 마감됐어요" (code 는 둘 다 `NOT_RECRUITING`). 자기 자신의 프로젝트인지 검사는 없다 (역할이 달라 애초에 불가).
- 성공 `201`

```
{ "ok": true, "data": { "application": { "id": "cm2a…", "status": "pending", "createdAt": "2026-10-21T01:00:00.000Z" } } }
```

- 실패: `401` · `403 ROLE_FORBIDDEN` · `428 PROFILE_REQUIRED` · `400 VALIDATION_FAILED` · `404 NOT_FOUND` · `409 NOT_RECRUITING` · `409 ALREADY_APPLIED` (Prisma `P2002`)
- 검증 순서: 역할 → 프로필 → 본문 → 상태 동기화 → 프로젝트 존재 → 모집 중 → 저장(중복 검사는 DB unique 로)
- 부수 효과: 본문 검증 뒤 `syncRecruitStatuses()` 실행(§3.8 과 같은 자동 전환). 저장은 트랜잭션: `Application` 생성 (`status: pending`) + 건물주에게 `application_received` 알림 1건 (`title: "<전문가명> 전문가가 지원했어요"`, `link: /projects/<id>/applicants`).

3.15 `POST /api/projects/[id]/bookmark` — 북마크 토글

- 호출: 전문가(freelancer)
- 요청 본문: 없음
- 성공 `200` — 있으면 지우고 `false`, 없으면 만들고 `true`

```
{ "ok": true, "data": { "bookmarked": true } }
```

- 실패: `401` · `403 ROLE_FORBIDDEN` · `404 NOT_FOUND` (없음/삭제됨)
- 부수 효과: `Bookmark` 행 생성/삭제. 모집 상태와 무관하게 가능. 트랜잭션 아님(조회→삭제/생성 2단계).

3.16 `POST /api/projects/[id]/cancel` — 프로젝트 취소

- 호출: 건물주(client), 본인 소유
- 요청 본문: 없음
- 규칙 (검사 순서) 1. 없음/삭제됨 → `404 NOT_FOUND` 2. 남의 것 → `403 FORBIDDEN` 3. `dealStatus === "cancelled"` → `409 ALREADY_CANCELLED` 4. 계약이 양측 서명됐거나(`Contract.clientSignedAt && freelancerSignedAt`), `dealStatus` 가 있고 `awaiting_contract` 가 아니면 → `409 CONTRACT_SIGNED`
- 즉 취소 가능한 상태 = `dealStatus` 가 `null` (수락 전) 또는 `awaiting_contract` (수락 후 서명 전).
- 성공 `200`

```
{ "ok": true, "data": { "projectId": "cm2p…", "notified": 3 } }
```

`notified` = 알림 받은 지원자 수 (`pending` + `accepted`).

- 실패: `401` · `403 ROLE_FORBIDDEN` · `404` · `403 FORBIDDEN` · `409 ALREADY_CANCELLED` · `409 CONTRACT_SIGNED`
- 부수 효과 (한 트랜잭션): `pending` 지원 → `rejected` (**`accepted` 지원은 상태를 바꾸지 않고 그대로 둔다**) → 프로젝트 `recruitStatus = closed`, `dealStatus = cancelled` → `pending` + `accepted` 지원자 전원에게 `project_cancelled` 알림. 취소된 프로젝트는 이후 `DELETE` 가능(`cancelled` 는 IN_DEAL 예외).

3.17 `POST /api/projects/[id]/close` — 모집 수동 마감

- 호출: 건물주(client), 본인 소유 + 미삭제
- 요청 본문: 없음
- 규칙: 이미 `closed` 면 `409 ALREADY_CLOSED`. `scheduled`(모집 전)도 바로 마감할 수 있다.
- 성공 `200`

```
{ "ok": true, "data": { "project": { "id": "cm2p…", "title": "옥상 방수 재시공", "recruitStatus": "clos
ed" } } }
```

- 실패: `401` · `403 ROLE_FORBIDDEN` · `404 NOT_FOUND` (남의 것도 404) · `409 ALREADY_CLOSED`
- 부수 효과 (한 트랜잭션, `closeRecruitingTx` — 자동 마감과 공용): `pending` 지원 전부 `rejected` → `recruitStatus = closed` → 거절된 지원자에게 `recruit_closed` 알림. `dealStatus` 는 건드리지 않는다.

3.18 `PATCH /api/applications/[id]` — 지원 수락 / 거절

- 호출: 건물주(client), 해당 지원서가 달린 프로젝트의 소유자
- 경로: `id` — 지원서(Application) id
- 요청 본문 (`decisionSchema`)

필드	타입	필수	허용값
`action`	string	O	`"accept"` | `"reject"`

- 규칙 (검사 순서, 공통): 지원서 없음 또는 프로젝트 삭제됨 → `404 NOT_FOUND` → 남의 프로젝트 → `403 FORBIDDEN` → `status !== pending` → `409 ALREADY_DECIDED`.
- `accept` 추가: 프로젝트 `dealStatus` 가 이미 있으면 `409 ALREADY_IN_DEAL`. 그 뒤 **조건부 갱신**(`where: { dealStatus: null }`, `where: { status: pending }`)으로 동시 요청 중 하나만 성공시킨다. 조건부 갱신이 0건이면 각각 `ALREADY_IN_DEAL` / `ALREADY_DECIDED`.
- 모집 상태(`open` / `closed`)는 검사하지 않는다. 마감 후에도 `pending` 이 남아 있다면(마감 시 일괄 거절되므로 실제로는 거의 없음) 수락 가능.
- 성공 `200` — accept

```
{ "ok": true, "data": { "action": "accept", "applicationId": "cm2a…", "autoRejected": 2 } }
```

- 성공 `200` — reject

```
{ "ok": true, "data": { "action": "reject", "applicationId": "cm2a…" } }
```

- 실패: `401` · `403 ROLE_FORBIDDEN` · `400 VALIDATION_FAILED` · `404 NOT_FOUND` · `403 FORBIDDEN` · `409 ALREADY_DECIDED` · `409 ALREADY_IN_DEAL`
- 부수 효과
- **accept** (한 트랜잭션, 5가지): 프로젝트 `recruitStatus = closed` + `dealStatus = awaiting_contract` → 해당 지원 `accepted` → 같은 프로젝트의 나머지 `pending` 전부 `rejected` → 수락된 전문가에게 `application_accepted` 알림 1건 → 자동 거절된 전문가 각각에게 `application_rejected_auto` 알림.
- **reject** (한 트랜잭션): 해당 지원 `rejected` → 그 전문가에게 `application_rejected` 알림 1건. 프로젝트 상태는 그대로.

3.19 `GET /api/me/applications` — 내 지원 현황

- 호출: 전문가(freelancer)
- 정렬: `createdAt desc`. 거절·수락 뒤에도, 프로젝트가 소프트 삭제된 뒤에도 행이 남는다.
- 성공 `200` — `Application` 전체 컬럼 + `project` 일부. 이 응답에는 `dealStatus` 가 포함된다(내부 화면).

```
{
  "ok": true,
  "data": {
    "applications": [
      {
        "id": "cm2a…",
        "projectId": "cm2p…",
        "freelancerId": "cm2…",
        "coverLetter": "옥상 전면 우레탄 …",
        "proposedAmount": 2800000,
        "proposedDays": 7,
        "status": "accepted",
        "createdAt": "2026-10-21T01:00:00.000Z",
        "project": {
          "id": "cm2p…",
          "title": "옥상 방수 재시공",
          "category": "누수·방수",
          "budget": 3000000,
          "deadlineAt": "2026-11-30T00:00:00.000Z",
          "recruitStatus": "closed",
          "dealStatus": "awaiting_contract",
          "deletedAt": null,
          "client": { "name": "김건물" }
        }
      }
    ]
  }
}
```

- 실패: `401` · `403 ROLE_FORBIDDEN`
- 부수 효과: 없음 (상태 동기화 호출 안 함)

3.20 `GET /api/me/bookmarks` — 내 북마크

- 호출: 전문가(freelancer)
- 조건: 프로젝트 `deletedAt: null` 인 것만. 정렬 `createdAt desc` (북마크 시각).
- 성공 `200` — `project` 는 §2.2 의 **평탄화 전** 카드 모양 그대로

```
{
  "ok": true,
  "data": {
    "bookmarks": [
      {
        "createdAt": "2026-10-22T09:00:00.000Z",
        "project": {
          "id": "cm2p…",
          "title": "옥상 방수 재시공",
          "category": "누수·방수",
          "budget": 3000000,
          "deadlineAt": "2026-11-30T00:00:00.000Z",
          "recruitStatus": "open",
          "createdAt": "2026-10-20T03:12:45.000Z",
          "skills": [{ "skill": { "name": "방수·도장" } }],
          "_count": { "applications": 2 },
          "client": { "name": "김건물", "clientProfile": { "region": "서울 마포구" } }
        }
      }
    ]
  }
}
```


- 실패: `401` · `403 ROLE_FORBIDDEN`
- 부수 효과: 없음 (상태 동기화 호출 안 함 — `recruitStatus` 가 목록보다 오래됐을 수 있음)

3.21 `GET /api/me/projects` — 내 프로젝트

- 호출: 건물주(client)
- 조건: `clientId = 나` , `deletedAt: null` . 정렬 `createdAt desc` . 거래 단계(`dealStatus`)는 이 응답에서 노출된다 (B-17).
- 성공 `200` — Prisma select 결과(`_count` , `applications`)가 spread 로 남고 가공 2개가 더해진다

```
{
  "ok": true,
  "data": {
    "projects": [
      {
        "id": "cm2p…",
        "title": "옥상 방수 재시공",
        "category": "누수·방수",
        "budget": 3000000,
        "deadlineAt": "2026-11-30T00:00:00.000Z",
        "recruitStatus": "closed",
        "dealStatus": "awaiting_contract",
        "createdAt": "2026-10-20T03:12:45.000Z",
        "_count": { "applications": 0 },
        "applications": [{ "freelancer": { "name": "박전문" } }],
        "pendingCount": 0,
        "acceptedFreelancer": "박전문"
      }
    ]
  }
}
```


- `_count.applications` 와 `pendingCount` 는 같은 값(**`pending` 상태만** 센 수. `GET /api/projects` 의 `applicationCount` 와 다름). `applications` 는 `accepted` 1건만(`take: 1`), 없으면 `[]` 이고 `acceptedFreelancer` 는 `null` .
- 실패: `401` · `403 ROLE_FORBIDDEN`
- 부수 효과: 조회 전 `syncRecruitStatuses()` 실행 (§3.8 과 동일, 전체 프로젝트 대상).

3.22 `GET /api/notifications` — 내 알림

- 호출: 로그인 (역할 무관)
- 최근 50건 (`createdAt desc` , `take: 50`). 읽은 것도 포함. `unreadCount` 는 50건 제한과 무관하게 전체 미확인 수.
- 성공 `200` — `Notification` 전체 컬럼

```
{
  "ok": true,
  "data": {
    "notifications": [
      {
        "id": "cm2n…",
        "userId": "cm2…",
        "type": "application_received",
        "title": "박전문 전문가가 지원했어요",
        "body": "옥상 방수 재시공",
        "link": "/projects/cm2p…/applicants",
        "readAt": null,
        "createdAt": "2026-10-21T01:00:00.000Z"
      }
    ],
    "unreadCount": 1
  }
}
```

- 실패: `401`

3.23 `POST /api/notifications` — 읽음 처리

- 호출: 로그인
- 요청 본문 (`readNotificationSchema`) — `id` 또는 `all` 중 하나는 truthy 여야 한다

필드	타입	필수	제약
`id`	string	X	1자 이상. 지정 시 그 알림 하나만
`all`	boolean	X	`true` 면 내 미확인 알림 전부

`{ "all": false }` 만 보내면 refine 에 걸려 `400 VALIDATION_FAILED` ("읽음 처리할 대상을 지정해 주세요"). `id` 와 `all: true` 를 같이 보내면 `id` 조건이 우선(where 에 id 가 붙음).

- 성공 `200`

```
{ "ok": true, "data": { "updated": 1, "unreadCount": 0 } }
```

- 남의 알림 id 나 이미 읽은 id 를 보내도 오류가 아니라 `updated: 0` 으로 200. (`where: { userId: 나, readAt: null, id }`)
- 실패: `401` · `400 VALIDATION_FAILED` · `400 INVALID_JSON`
- 부수 효과: `readAt = now` .

3.24 `GET /api/notifications/[id]/open` — 알림 클릭

- 호출: 누구나. **JSON 이 아니라 리다이렉트**로 응답한다 (일반 `<a href>` 로 동작하게 만든 것, JS 불필요).
- 경로: `id` — 알림 id

- 응답 (`NextResponse.redirect` , 기본 상태 코드 307)

상황	Location
비로그인	`/login`
내 알림이 아니거나 없음	`/notifications`
정상	알림의 `link` (예: `/projects/<id>/applicants` , `/me/applications`)

- 부수 효과: 미확인이면 `readAt = now` . 이미 읽었으면 갱신 없이 이동만. 봉투(`ok`) 형식을 쓰지 않으며 `fail()` 도 거치지 않는다 — DB 오류 시 Next 기본 500.

부록

A.1 enum — `RecruitStatus` (모집 단계, 외부 노출)

값	뜻	진입 조건
`scheduled`	모집 예정	등록/수정 시 `recruitStartAt` 이 미래
`open`	모집 중	등록/수정 시 시작일 없음·과거, 또는 `sync` 에서 시작일 도래
`closed`	모집 마감	마감일 경과(`sync`) · 수동 마감 · 지원 수락 · 취소 · 삭제

스키마 기본값은 `scheduled` 이나 `POST /api/projects` 가 항상 명시 지정한다.

A.2 enum — `DealStatus` (거래 단계, 내부 전용 · `null` 허용)

값	뜻	현재 API 에서의 진입
`null`	거래 전	등록 직후
`awaiting_contract`	계약 대기 (금액 합의·서명)	`PATCH /api/applications/[id]` accept
`awaiting_payment`	결제 대기	**진입 API 없음 — 미구현 (코드 확인 필요)**
`in_progress`	작업 중	동상
`completed`	완료	동상
`cancelled`	취소	`POST /api/projects/[id]/cancel`

A.3 enum — `ApplicationStatus`

값	뜻
`pending`	검토중 (생성 기본값)
`accepted`	수락
`rejected`	거절 — 직접 거절 · 타 지원 수락에 따른 자동 거절 · 마감/취소/삭제 일괄 거절 모두 같은 값

A.4 알림 `type` (`NOTIFICATION_TYPES` , DB 컬럼은 String)

type	수신자	트리거	title	body	link
`application_received`	건물주	지원 제출	`<전문가명> 전문가가 지원했어요`	프로젝트 제목	`/projects/<id>/applicants`
`application_accepted`	수락된 전문가	accept	`지원이 수락됐어요`	`<제목> — 다음 단계는 범위와 금액 합의입니다`	`/me/applications`
`application_rejected_auto`	자동 거절된 전문가	accept (다른 지원자)	`다른 전문가가 선정됐어요`	제목	`/me/applications`
`application_rejected`	거절된 전문가	reject	`지원이 거절됐어요`	제목	`/me/applications`

type	수신자	트리거	title	body	link
`recruit_closed`	`pending` 이던 지원자 전원	수동 마감 · 마감일 자동 마감	`모집이 마감됐어요`	제목	`/me/applications`
`project_cancelled`	취소: `pending` + `accepted` 지원자 / 삭제: `pending` 지원자	cancel · DELETE	`프로젝트가 취소됐어요`	제목	`/me/applications`

알림은 모두 상태 변경과 같은 트랜잭션 안에서 생성된다 (`notifyMany(tx, …)`).

A.5 기타 고정값

항목	값	출처
페이지 크기	10	`PAGE_SIZE`
알림 목록 상한	50	`GET /api/notifications`
`buildingType`	상가 / 오피스 / 다세대 / 기타	`clientProfileSchema`
`PROJECT_CATEGORIES` (상수, 서버 미강제)	누수·방수 / 전기·조명 / 설비·배관 / 외벽·도장 / 인테리어·목공 / 소방·안전 / 공조·환기 / 기타	`validators.project.ts`
시드 전문분야 (Skill)	전기 / 설비·배관 / 방수·도장 / 인테리어·목공 / 소방·안전 / 공조·환기	`prisma/seed.ts`
`UserRole`	`client` (건물주) / `freelancer` (전문가)	schema

A.6 상태 전이 요약

프로젝트 (recruitStatus / dealStatus)

트리거	recruitStatus	dealStatus	지원서	알림
등록 (`POST /api/projects`)	`open` 또는 `scheduled`	`null`	-	-
수정 `recruitStartAt` (closed 아닐 때)	재계산 `open` / `scheduled`	-	-	-

트리거	recruitStatus	dealStatus	지원서	알림
`sync` : 시작일 도래	`scheduled → open`	-	-	-
`sync` : 마감일 경과	`* → closed`	-	pending → rejected	`recruit_closed`
수동 마감 (`/close`)	`scheduled/open → closed`	-	pending → rejected	`recruit_closed`
지원 수락 (`accept`)	`→ closed`	`null → awaiting_contract`	1건 accepted, 나머지 pending → rejected	`application_accepted` ×1, `application_rejected_auto` ×N
지원 거절 (`reject`)	-	-	1건 pending → rejected	`application_rejected`
취소 (`/cancel`)	`→ closed`	`null/awaiting_contract → cancelled`	pending → rejected (accepted 유지)	`project_cancelled` (pending+accepted)
삭제 (`DELETE`)	`→ closed` + `deletedAt`	변경 없음 (`null` / `cancelled` 만 허용)	pending → rejected	`project_cancelled` (pending)

`sync` 가 호출되는 시점: `GET /api/projects` , `GET /api/me/projects` , `POST /api/projects/[id]/applications` . 나머지 라우트는 저장된 값을 그대로 쓴다.

지원서

```
pending ──accept──▶ accepted
pending ──reject / 타 지원 accept / close / cancel / delete / sync 마감──▶ rejected
accepted ──(cancel 해도 유지)
```

`accepted` · `rejected` 에서 되돌리는 API 는 없다. 같은 프로젝트에 재지원도 불가(행이 남아 unique 충돌 → `ALREADY_APPLIED`).